Fundamentals of Data Science

Master's Degree in Data Science, Sapienza Università di Roma
Faculty of Information Engineering, Informatics and Statistics

Department of Computer Science

Author: Gianmaria Romano

Reference Bibliography

M. P. Deisenroth, A. A. Faisal, C. S. Ong: *Mathematics for Machine Learning*
S. Prince: *Understanding Deep Learning*
J. Watt, R. Borhani, A. K. Katsaggelos: *Machine Learning Refined*
W. McKinney: *Python for Data Analysis*
M. E. J. Newman: *Networks: An Introduction*

Academic Year 2026/2027

Disclaimer

This document contains notes for the **Fundamentals of Data Science** course, given by **Professors Cinelli and Spinelli** as part of the Master's Degree in Data Science at **Sapienza Università di Roma** during the **first** semester of Academic Year **2026/2027**.

Note: These notes may not cover all details of each lecture, so it is highly suggested to consult the Professor's official resources and materials.

Important: These notes are free to share and use, but please remember to credit me as the author and do not try to hide/remove this page.

Contacts

For further information or requests, feel free to contact me privately or reach out through my repository: https://github.com/GianmariaRomano/Data-Science-Notes.

License

These documents are distributed under the GNU General Public License 3.0, a form of copyleft intended for use on a manual, textbook or other documents. Material licensed under the current version of the license can be used for any purpose, as long as the use meets certain conditions:

- All previous authors of the work must be attributed.
- All changes to the work must be logged.
- All derivative works must be licensed under the same license.
- The full text of the license, unmodified invariant sections as defined by the author if any, and any other added warranty disclaimers (such as a general disclaimer alerting readers that the document may not be accurate for example) and copyright notices from previous versions must be maintained.
- Technical measures such as DRM may not be used to control or obstruct distribution or editing of the document.

Contents

1 Introduction to data science **5**
- 1.1 The lifecycle of data science . . . 5
- 1.2 The role of machine learning in data science . . . 5
- 1.3 Supervised learning . . . 6
- 1.4 Unsupervised learning . . . 6
- 1.5 Reinforcement learning . . . 6

2 Vectors **8**
- 2.1 Representing data using vectors . . . 8
- 2.2 Spanning sets and bases . . . 8
  - 2.2.1 Standard bases . . . 9
  - 2.2.2 Non-standard bases . . . 9
- 2.3 Vector subspaces . . . 10

3 Optimization methods **12**
- 3.1 The goal of optimization . . . 12
  - 3.1.1 Zero-order optimization . . . 12
  - 3.1.2 First-order optimization . . . 12
- 3.2 Hyperplanes . . . 13
- 3.3 Local linear approximation . . . 13
- 3.4 Automatic differentiation . . . 14
- 3.5 Gradient descent . . . 15
  - 3.5.1 Zig-zagging gradients . . . 17
  - 3.5.2 Slow-crawling . . . 17

I Machine Learning **18**

4 Principal component analysis **19**
- 4.1 Linear transformations . . . 19
- 4.2 Learning the projection basis . . . 19
- 4.3 Optimizing a single basis vector . . . 20
  - 4.3.1 Eigenvalue decompositions . . . 22
- 4.4 Applications of principal component analysis . . . 22

5 Linear models **23**
5.1 The risk minimization problem in supervised learning . . . 23
5.2 Linear regression . . . 23
5.2.1 Alternative performance metrics . . . 25
5.2.2 Vectorized linear regression . . . 25
5.3 Polynomial regression . . . 26
5.4 Linear classification models . . . 27
5.4.1 Logistic regression . . . 27
5.4.2 Multi-class logistic regression . . . 29
5.5 Classification metrics . . . 30
5.5.1 The accuracy of a classification model . . . 30
5.5.2 The sensitivity of a classification model . . . 30
5.5.3 The specificity of a classification model . . . 31
5.5.4 The F1 score of a classification model . . . 31
5.5.5 The receiving operating characteristic curve of a classification model . . . 31
5.6 The bias-variance trade-off . . . 31
5.7 Model regularization . . . 32
5.7.1 Ridge regression . . . 33
5.7.2 Lasso regression . . . 33

6 Non-parametric models **34**
6.1 The non-parametric learning paradigm . . . 34
6.2 The K-nearest neighbours algorithm . . . 34
6.3 Decision trees . . . 35
6.3.1 Using decision trees for regression tasks . . . 36
6.4 Ensemble learning techniques . . . 36
6.4.1 Bagging techniques . . . 37
6.4.2 Random forests . . . 37
6.4.3 Boosting models . . . 37

7 Neural networks **38**
7.1 The representation learning paradigm . . . 38
7.2 Shallow neural networks . . . 39
7.2.1 Optimizing a shallow neural network using backpropagation . . . 40

II Network Science **41**

8 Understanding networks **42**
8.1 The purpose of network science . . . 42
8.2 A formal definition of networks . . . 43
8.2.1 Visualizing a network using an edge list . . . 43
8.2.2 Visualizing a network using an adjacency matrix . . . 44
8.2.3 Visualizing a network using an adjacency list . . . 44
8.3 Neighbourhoods and subgraphs . . . 44
8.4 Network measures . . . 45
8.4.1 Network density . . . 45

8.4.2 The degree of a node . . . 46
8.5 Weighted networks . . . 46
8.5.1 The strength of a node . . . 47
8.6 The degree distribution of a network . . . 47
8.7 Bipartite networks . . . 48
8.7.1 The density of a bipartite network . . . 49
8.7.2 Performing one-mode projection on a bipartite network . . . 49
8.8 Paths in networks . . . 50
8.8.1 The average path length . . . 51
8.8.2 Connectedness in networks . . . 52
8.8.3 Random walks . . . 52
8.9 Network descriptives . . . 52
8.9.1 The clustering coefficient . . . 52
8.9.2 The centrality of a node . . . 53

9 Random networks **54**
9.1 The purpose of random graph models . . . 54
9.2 The Erdős-Rényi model . . . 54
9.2.1 The edge and degree distribution of an Erdős-Rényi network . . . 55
9.2.2 Analysing components in an Erdős-Rényi network . . . 55

Chapter 1

Introduction to data science

1.1 The lifecycle of data science

Data science can be seen as an interdisciplinary field that focuses on extracting actionable insights and knowledge from structured and unstructured data using techniques based on statistical and mathematical methods, as well as computer science algorithms and domain-specific knowledge.
Generally speaking, a data science pipeline starts with understanding the business objectives and translating them into machine learning objectives.
Next, after collecting and cleaning data, the pipeline focuses on selecting and training a model and evaluating the results for deployment, ultimately monitoring model efficacy and restarting the pipeline, either by collecting new data or re-training the model, whenever needed.

1.2 The role of machine learning in data science

Artificial intelligence is a sub-field of computer science that focuses on creating agents that can perform tasks requiring human intelligence and problem solving.
More specifically, Tom Mitchell defines machine learning as "the study of algorithms that improve their performance P at some task T with experience E", typically focusing on tasks that would be too complex to just hard-code.

Example: Playing checkers

The task $\langle P, T, E \rangle$ to train an agent to play checkers can be defined using the following components:

- The agent's performance P is evaluated as the proportion of matches it wins against an arbitrary opponent.
- The task T to perform is to train the model to make the correct moves during a match.
- The experience E used to train the agent comes from matches the agent plays against itself.

While machine learning and statistics typically use the same instruments and algorithms to discover patterns within data, the two fields are normally considered to be different areas because statistics follows a more rigorous inference-based approach, while machine learning focuses on making predictions using scalable and autonomous techniques.

1.3 Supervised learning

In a supervised learning task, the model is given some input examples $(\mathbf{x}, y)$ with the aim of learning a function $\hat{y} = h(\mathbf{x})$ that will be used to predict the target variable based on new, unseen data.
In particular, a supervised learning task can be formulated as a classification problem, which focuses on prediction discrete classes based on a probability distribution, or as a regression problem, which aims to predict continuous values.

Example: Some supervised learning tasks

Common classification tasks can include binary classification on the sentiment of a review or multi-class classification of which cat breed is featured in an image.
On the other hand, regression tasks can include univariate prediction of house prices or bivariate prediction of the freezing and boiling points of a chemical.

1.4 Unsupervised learning

In an unsupervised task, a model uses some input data $\mathbf{x}$ to discover patterns and structural relationships or to generate new data samples.

Example: Generative models as unsupervised learning

Generative models, such as generative adversarial networks or (variational) autoencoders, are considered to be an example of unsupervised learning because they learn how to create new data by sampling information from a distribution of unlabeled data.

1.5 Reinforcement learning

A reinforcement task is defined using a sequence of states, actions and rewards that the agent uses to find a policy that allows it to maximize its overall reward.
In this setting, the main challenges come from the stochastic nature of the task, which means that the environment might react differently to the same action, and the trade-off between exploitation and exploration as the agent needs to determine whether to stick to a known good policy or try out new actions in the hopes of obtaining a better reward.

Example: Chess as a reinforcement learning task

It is possible to train an agent to play chess using reinforcement learning.
In fact, the idea is to consider the task as a configuration (S, A, R) in which S represents the set of possible valid states and A represents the set of possible legal moves, while the reward R will be positive whenever the agent manages to take the opponent's pieces and negative whenever the agent loses its pieces.

Chapter 2

Vectors

2.1 Representing data using vectors

When dealing with supervised learning tasks, a dataset consists of measurements of some features of objects, which are used to predict a target variable.
However, while features can be represented as numerical, categorical or binary variables, most machine learning algorithms require numerical inputs.
For this reason, categorical variables are often transformed into numerical representations through one-hot encoding, which, in the case of mutually exclusive classes, represents each feature using a vector of type $\mathbf{e}_i = (0, \dots, 1, \dots, 0)^T$, where the position of the single non-zero entry identifies the class to which the observation belongs.

Example: The IRIS Dataset

Fisher's IRIS Dataset is a toy example of a dataset that classifies each flower into one of three possible subspecies based on petal and sepal measurements.

From a mathematical perspective, a dataset can be described as a set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{n} \subset (X \times Y)^n$, where $X \subseteq \mathbb{R}^p$ is known as the "input space" while Y is known as the target space, or as a matrix whose rows represent single observations and whose columns denote specific features.
In particular, it is assumed that the dataset is generated from an unknown distribution $P_{X,Y}$ on $X \times Y$, which the model aims to (partially) learn, and that each observed sample $(\mathbf{x}^{(i)}, y^{(i)})$ is drawn independent and identically distributed from $P_{X,Y}$, which means that all samples are mutually independent and come from the same probability distribution.

2.2 Spanning sets and bases

Assuming a dataset has been mean-centered, each observed sample can be seen as a data point in the corresponding input vector space.
From a broader perspective, a collection of vectors is said to form a spanning set if combining the vectors using addition and scalar multiplication allows to create every vector in the corresponding vector space.

In particular, a basis is any minimal spanning set that, having no redundant vectors, satisfies the condition of linear independence.

Example: Spanning sets in $\mathbb{R}^2$

In the vector space $\mathbb{R}^2$, a possible spanning set can be composed using the canonical vectors $\mathbf{e}_1 = (1,0)^T$ and $\mathbf{e}_2 = (0,1)^T$ as each point $(x,y) \in \mathbb{R}^2$ can be reformulated as $x\mathbf{e}_1 + y\mathbf{e}_2$. Additionally, it is possible to observe that this spanning set also works as a basis for $\mathbb{R}^2$ because its vectors are linearly independent.

Generally speaking, a candidate basis $\mathbf{c}_1, \ldots, \mathbf{c}_N \in \mathbb{R}^N$ is able to perfectly represent a dataset $\{\mathbf{x}_i\}_{p=1}^P \in \mathbb{R}^N$ if it is possible to find some weights such that

$$\sum_{n=1}^{N} \mathbf{c}_n w_{n,p} = \mathbf{x}_p \ \forall p \in \{1, \ldots, P\}$$

2.2.1 Standard bases

The canonical vectors $\mathbf{e}_1, \ldots, \mathbf{e}_N \in \mathbb{R}^N$ always form a standard basis such that

$$\mathbf{e}_n^T \mathbf{e}_m = 0 \ \forall n \neq m \text{ and } \|\mathbf{e}_n\|_2^2 = 1 \ \forall n \in \{1, \ldots, N\}$$

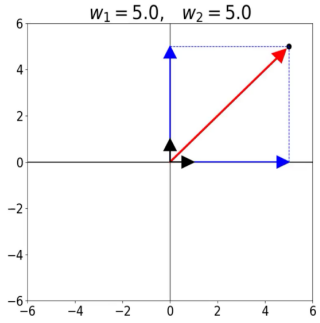


Figure 2.1: Example of a standard basis in $\mathbb{R}^2$.

In this case, representing a vector $\mathbf{x}_p$ is a trivial task as the encoding weights will always be given by $\mathbf{w}_p = \mathbf{x}_p$.
It should be noticed that all these properties can also be expressed using the concatenated basis matrix $\mathbf{C}$: knowing that $\mathbf{C}^T\mathbf{C} = \mathbf{C}\mathbf{C}^T = \mathbf{I}_{N\times N}$, it is in fact possible to find that

$$\mathbf{w}_p = \mathbf{C}^T\mathbf{x}_p \text{ and } \mathbf{C}\mathbf{C}^T\mathbf{x}_p = \mathbf{x}_p \ \forall p \in \{1, \ldots, P\}$$

2.2.2 Non-standard bases

When dealing with a non-standard basis, the new vector $\mathbf{w}_p$ acts as the new encoding of $\mathbf{x}_p$ in the transformed feature space whose coordinate axes are defined by the spanning set.

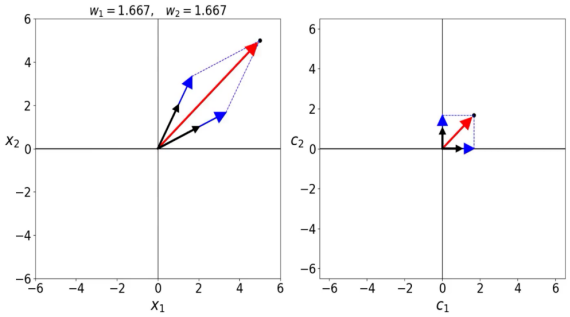


Figure 2.2: Example of a non-standard basis in $\mathbb{R}^2$.

Therefore, using the concatenated basis matrix, the goal is to find the encoding that solves the linear system of equations

$$\mathbf{C}^T\mathbf{C}\mathbf{w}_p = \mathbf{C}^T\mathbf{x}_p \ \forall p \in \{1, \dots, P\}, \text{ which is equivalent to solving}$$

$$\min_{\mathbf{w}_1,\dots,\mathbf{w}_P} \frac{1}{P}\sum_{p=1}^{P} \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2, \text{ which represents the average reconstruction error.}$$

Example: A non-standard basis in $\mathbb{R}^2$

Consider a non-standard basis in $\mathbb{R}^2$ given by

$$\mathbf{c}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \text{ and } \mathbf{c}_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Expressing a vector using the equation $\mathbf{x}_p = w_{1,p}\mathbf{c}_1 + w_{2,p}\mathbf{c}_2$ reveals that this basis corresponds to a transformation that consists of a rotation followed by a vector shearing.

2.3 Vector subspaces

When representing N-dimensional data points $\{\mathbf{x}_p\}_p^P \in \mathbb{R}^N$ using $K \leq N$ basis vectors, the representation incurs in a loss of information because the data points are projected onto a lower-dimensional subspace.

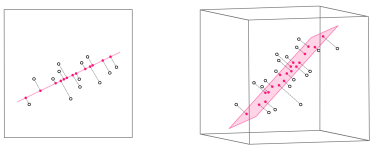


Figure 2.3: Projecting data points from $\mathbb{R}^3$ onto 1D and 2D subspaces.

In order to minimize the reconstruction error, each data point is projected orthogonally onto the subspace spanned by the basis vectors, meaning that the encoding weights will be computed by solving the linear system of equations

$$\mathbf{C}^T\mathbf{C}\mathbf{w}_p = \mathbf{C}^T\mathbf{x}_p, \text{ where } \mathbf{C} \in \mathbb{R}^{N\times K}.$$

In fact, this strategy ensures that, during decoding, the projected point provides the best possible approximation of the starting data point within the subspace.

Example: Projecting a vector from $\mathbb{R}^2$ onto a line

Suppose you want to project the vector $\mathbf{x} = (3,1)^T$ onto the one-dimensional subspace spanned by the basis vector $\mathbf{c} = (1,1)^T$.
Since the basis vector is not a unit vector, the encoding weight must be computed as

$$\mathbf{c}^T\mathbf{c}w = \mathbf{c}^T\mathbf{x} \Rightarrow \begin{pmatrix}1 & 1\end{pmatrix}\begin{pmatrix}1\\1\end{pmatrix} w = \begin{pmatrix}1 & 1\end{pmatrix}\begin{pmatrix}3\\1\end{pmatrix}$$
$$\Rightarrow 2w = 4 \Rightarrow w = 2$$

Therefore, the reconstructed projection will be given by $w\mathbf{c} = (2,2)^T$, resulting in a residual reconstruction error of

$$r = \|\mathbf{x} - w\mathbf{c}\|_2 = \left\| \begin{pmatrix}3\\1\end{pmatrix} - \begin{pmatrix}2\\2\end{pmatrix} \right\|_2 = \left\| \begin{pmatrix}1\\-1\end{pmatrix} \right\|_2 = \sqrt{2}$$

Whenever the chosen basis is orthonormal, the weight encodings can be computed using

$$\mathbf{w}_p = \mathbf{C}^T\mathbf{x}_p \text{ and } \mathbf{C}\mathbf{C}^T\mathbf{x}_p \approx \mathbf{x}_p \ \forall p \in \{1,\dots,P\}$$

Chapter 3

Optimization methods

3.1 The goal of optimization

Given an objective function $g(\mathbf{w})$, optimization deals with finding the value $\mathbf{w}^*$ that minimizes or maximizes the function, which means that

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} g(\mathbf{w}) \text{ or } \mathbf{w}^* = \arg\max_{\mathbf{w}} g(\mathbf{w})$$

Additionally, maximizing an objective function $g(\mathbf{w})$ is always equivalent to minimizing $-g(\mathbf{w})$.

3.1.1 Zero-order optimization

Since zero-order optimization simply involves analysing the values taken by the objective function, it is possible to distinguish two types of solution:

- **Global optimum:** A point $\mathbf{w}^*$ is a global optimum if it represents a global maximum/minimum among every possible value of $\mathbf{w}$.
- **Local optimum:** A point $\mathbf{w}^*$ is a local optimum if it acts as a local maximum/minimum within some neighbourhood, but not necessarily among every possible value of $\mathbf{w}$.

Example: Zero-order optimization on a sinusoidal function

Consider the scalar function $g(w) = \sin(w)$: since this function has period equal to 2π, it is possible to observe that it has infinitely many global maxima and global minima.
More specifically the points $w^* = \frac{3\pi}{2} + 2k\pi$, with $k \in \mathbb{Z}$, act as global minima for the function, whereas the points $w^* = \frac{\pi}{2} + 2k\pi$, with $k \in \mathbb{Z}$, act as global maxima for the function.

3.1.2 First-order optimization

First-order optimization involves using (partial) derivatives to find candidate optima by exploiting the fact that, at an extremum of a differentiable function, it is always the case that

$$g'(w^*) = 0 \text{ or, for a multivariate function, } \nabla g(\mathbf{w}^*) = \mathbf{0}_{n\times 1}$$

Example: First-order optimization on a sinusoidal function

Consider the scalar function $g(w) = \sin(w)$: solving $g'(w) = \cos(w) = 0$ reveals that the points $w^* = \frac{\pi}{2} + k\pi$, with $k \in \mathbb{Z}$, act as extrema for the function.

Using the second derivative $g''(w) = -\sin(w)$ reveals that the points $w^* = \frac{3\pi}{2} + 2k\pi$ act as global minima for the function, whereas the points $w^* = \frac{\pi}{2} + 2k\pi$ act as global maxima for the function.

3.2 Hyperplanes

A hyperplane can be seen as a N-dimensional generalization of a line that can be defined as

$$h(\mathbf{w}) = a + \mathbf{b}^T\mathbf{w}$$

Since any point $\mathbf{w}_0$ admits infinitely many direction vectors $\mathbf{d}$, it is possible to find the direction of maximum increase or decrease by evaluating

$h(\mathbf{w}_0 + \mathbf{d}) = a + \mathbf{b}^T(\mathbf{w}_0 + \mathbf{d}) = a + \mathbf{b}^T\mathbf{w}_0 + \mathbf{b}^T\mathbf{d}$, which means that

$$\max_{\mathbf{d}} h(\mathbf{w}_0 + \mathbf{d}) \text{ subject to } \|\mathbf{d}\|_2^2 = 1 \equiv \max_d \mathbf{b}^T\mathbf{d} = \max_d \|\mathbf{b}\|_2^2\|\mathbf{d}\|_2^2\cos(\theta) \overset{\|\mathbf{d}\|=1}{=} \max_\theta \cos(\theta)$$

Solving this problem reveals that the maximum increase happens whenever $\theta = 0$, which corresponds to the direction vector $\mathbf{d} = \frac{\mathbf{b}}{\|\mathbf{b}\|_2^2}$, whereas the maximum decrease happens whenever $\theta = \pi$, which corresponds to the direction vector $\mathbf{d} = -\frac{\mathbf{b}}{\|\mathbf{b}\|_2^2}$.

3.3 Local linear approximation

The gradient of a multivariate function $g(\mathbf{w})$ is a vector $\nabla g(\mathbf{w})$ that contains all the partial derivatives of the function: this vector points towards the direction of maximum ascent, with its magnitude representing the rate of increase in this direction.

Since the gradient vector provides information about ascent and descent, it is possible to approximate the direction of steepest descent of a function g at a point $\mathbf{w}_0$ using the tangent hyperplane

$$g(\mathbf{w}) \approx h(\mathbf{w}) = g(\mathbf{w}_0) + \nabla g(\mathbf{w}_0)^T(\mathbf{w} - \mathbf{w}_0)$$

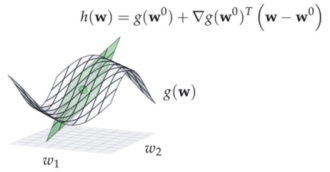


Figure 3.1: Visualization of the tangent hyperplane of a function.

It should be noticed that, whenever analytical differentiation is not feasible, derivatives can be approximated using the "central difference" method used in numerical differentiation, resulting in

$$g'(w_0) = \frac{g(w_0 + \varepsilon) - g(w_0 - \varepsilon)}{2}, \text{ for a small } \varepsilon > 0.$$

3.4 Automatic differentiation

A computation graph allows to simplify programmatic computations by decomposing a function $g(\mathbf{w})$ into a combination of elementary functions, which are easier to differentiate, and operations.

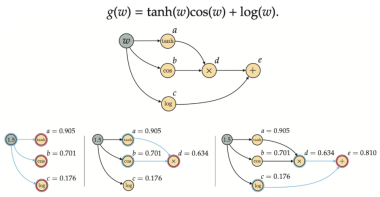


Figure 3.2: A computational graph for the function $g(w) = \tanh(w)\cos(w) + \log(w)$.

Using "forward mode", the graph is traversed left-to-right and each node computes both its value and its gradient with respect to the starting input, providing a simple method for differentiation.

Example: Running the forward method in a computation graph.

Given the function $g(w) = \tanh(w)\cos(w) + \log(w)$, define the following nodes:

$$a = \tanh(w) \quad b = \cos(w) \quad c = \log(w) \quad d = ab \quad e = c + d$$

For a starting value w, exploit $\dfrac{dw}{dw} = 1$ to compute the gradient of the child nodes:

$$\frac{da}{dw} = 1 - \tanh^2(w) \quad \frac{db}{dw} = -\sin(w) \quad \frac{dc}{dw} = \frac{1}{w}$$

Using this information, it is possible to compute the remaining gradients as well.

$$\begin{aligned}
\frac{dd(a,b)}{dw} &= \left(\frac{\partial d(a,b)}{\partial a}\frac{da}{dw}\right) + \left(\frac{\partial d(a,b)}{\partial b}\frac{db}{dw}\right) = \frac{da}{dw}b + a\frac{db}{dw} \\
&= (1 - \tanh^2(w)) - \tanh(w)\sin(w) \\
\frac{de(d(w),c(w))}{dw} &= \left(\frac{\partial e(d(w),c(w))}{\partial d}\frac{dd(w)}{dw}\right) + \left(\frac{\partial e(d(w),c(w)}{\partial c}\frac{dc(w)}{dw}\right) \\
&= ((1 - \tanh^2(w))\cos(w) - \tanh(w)\sin(w)) + \frac{1}{w}
\end{aligned}$$

The main issue of performing forward mode automatic differentiation lies in the fact that each node computes the gradient with respect to all inputs, leading to massive computational waste when working on large networks.
For this reason, it is more convenient to use "reverse mode" automatic differentiation, which consists of a forward pass, where each node uses the input value(s) to compute just the relevant partial derivatives, followed by a backward pass, where gradients are computed by recursively applying the chain rule from the output node.

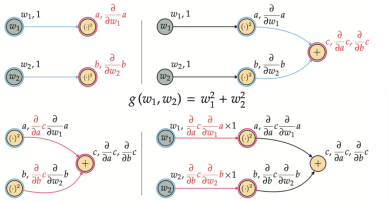


Figure 3.3: Reverse mode automatic differentiation of the function $g(w_1, w_2) = w_1^2 + w_2^2$.

Overall, performing reverse mode automatic differentiation allows to significantly reduce computation times, making this strategy useful for developing machine learning models, although memory overhead needs to be considered.

3.5 Gradient descent

Gradient descent exploits the fact that the negative gradient always represents a descent direction to find the global minimum of a function $g(\mathbf{w})$ using the following sequence:

$$\mathbf{w}_k = \mathbf{w}_{k-1} - \alpha \nabla g(\mathbf{w}_{k-1})$$

This means that, for an appropriate learning rate α, the model uses the current gradient to move towards the direction of steepest descent.

Example: Optimizing principal component analysis using gradient descent

Principal component analysis is an unsupervised learning algorithm that is commonly used to compress high-dimensional data onto a lower-dimensional subspace while retaining data variability in order to preserve information.
While the algorithm can be optimized by maximizing data variance, one can also apply gradient descent to minimize the loss function, resulting in

$$\begin{aligned} \min_{\mathbf{C}} L(\mathbf{C}) &= \|\mathbf{C}\mathbf{C}^T\mathbf{X} - \mathbf{X}\|_2^2 \text{ subject to } \mathbf{C}^T\mathbf{C} = \mathbf{I} \\ \nabla_{\mathbf{C}} L(C) &= 2(\mathbf{C}\mathbf{C}^T\mathbf{X} - \mathbf{X})\mathbf{X}^T\mathbf{C} \\ \mathbf{C}_k &= \mathbf{C}_{k-1} - \alpha \nabla_{\mathbf{C}} L(\mathbf{C}), \text{ with } \mathbf{C}^T\mathbf{C} = \mathbf{I}. \end{aligned}$$

While easy to implement, gradient descent is only guaranteed to find a global minimum if g is a convex function as its local minima correspond to the global minima.

In fact, if g is not a convex function, performance depends on the initialization step and learning rate because a small learning rate severely slows down computations, whereas a large learning rate might result in gradient divergence.
For this reason, it can be convenient to monitor gradient descent using a cost function history plot, which plots how the value of a function changes as the algorithm is executed, or by looking at the function's contours.

Example: Gradient descent on convex and non-convex functions

For the convex function $g(w_1, w_2) = w_1^2 + w_2^2 + 2$, the gradient descent algorithm always converges to the global minimum, regardless of the initialization setting.

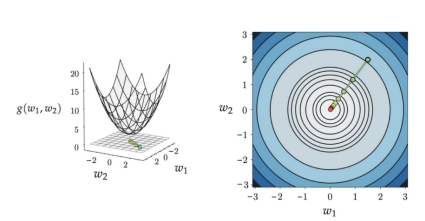


Figure 3.4: Visualization of gradient descent for the function $g(w_1, w_2) = w_1^2 + w_2^2 + 2$.

By contrast, for the function $g(w) = \sin(3w) + 0.3w^2$, the algorithm's performance does depend on the initialization step because, for instance, choosing $w_0 = -1.5$ allows the algorithm to find a global minimum, whereas starting from $w_0 = 4.5$ causes the algorithm to stop at a local minimum.

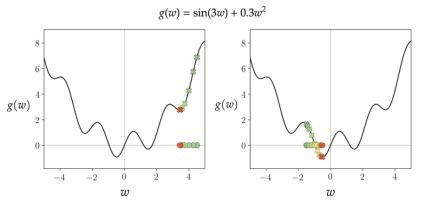


Figure 3.5: Visualization of gradient descent for the function $g(w) = \sin(3w) + 0.3w^2$.

Overall, it is known that the gradient descent algorithm terminates near the stationary points of a function, where

$$\mathbf{w}_k = \mathbf{w}_{k-1} - \alpha \nabla g(\mathbf{w}_{k-1}) \text{ results in } \mathbf{w}_k \approx \mathbf{w}_{k-1} \Rightarrow \nabla g(\mathbf{w}_{k-1}) \approx 0$$

For this reason, it is possible to choose the halting criterion among the following options:

1. **Gradient-based criterion:** $\|\nabla g(\mathbf{w}_{k-1})\|_2$ is sufficiently small to suggest that a stationary point has been reached.

2. **Step-based criterion:** No significant progress is observed between steps, meaning that, for some small $\varepsilon > 0$, it is the case that

$$\frac{1}{N}\|\mathbf{w}_k - \mathbf{w}_{k-1}\|_2 \leq \varepsilon$$

3. **Function value criterion:** The cost function changes negligibly between steps, meaning that, for some small $\varepsilon > 0$, it is the case that

$$\frac{1}{N}\|g(\mathbf{w}_k) - g(\mathbf{w}_{k-1})\|_2 \leq \varepsilon$$

4. **Practical criterion:** The algorithm is terminated after a chosen number of iterations.

3.5.1 Zig-zagging gradients

Whenever the model enters a region where the contours of g are (nearly) parallel, the gradient will start oscillating between steps, causing convergence to slow down.

3.5.2 Slow-crawling

Whenever the model gets close to a stationary point, the gradient descent procedure slows down because the updates are not significant anymore.
This situation becomes problematic whenever g is not a convex function because the algorithm may terminate at a wrong point, such as a local minimum or a saddle point, instead of reaching the global minimum.

Part I

Machine Learning

Chapter 4

Principal component analysis

4.1 Linear transformations

Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, a linear transformation $\mathbf{y} = \mathbf{A}\mathbf{x}$ maps a vector $\mathbf{x} \in \mathbb{R}^n$ to a vector $\mathbf{y} \in \mathbb{R}^m$.

- **Scaling:** If $\mathbf{A}$ is a diagonal matrix, then the transformation $\mathbf{y} = \mathbf{A}\mathbf{x}$ simply stretches or shrinks each axis by the corresponding factor.
- **Rotation:** If $\mathbf{A}$ is an orthonormal matrix, meaning that $\mathbf{A}^T\mathbf{A} = \mathbf{I}$, then the transformation $\mathbf{y} = \mathbf{A}\mathbf{x}$ represents a rotation or a reflection.
- **Shearing:** If $\mathbf{A}$ features some off-diagonal entries, then the transformation $\mathbf{y} = \mathbf{A}\mathbf{x}$ typically results in a slanted vector.
- **Projection:** If $\mathbf{A}$ has rank $k < n$, then the transformation $\mathbf{y} = \mathbf{A}\mathbf{x}$ always results in a loss of information because the vectors collapse into a lower-dimensional space.

4.2 Learning the projection basis

Rather than finding the encoding weights for a given basis, principal component analysis aims to learn both the encoding weights $\mathbf{w}_1, \ldots, \mathbf{w}_p$ and the basis matrix $\mathbf{C}$ that minimize the mean reconstruction error

$$g(\mathbf{w}_1, \ldots, \mathbf{w}_p, \mathbf{C}) = \frac{1}{P} \sum_{p=1}^{P} \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2, \text{ for a dataset } \{\mathbf{x}_p\}_{p=1}^P.$$

However, if $\mathbf{C}$ is assumed to be orthonormal, it is possible to directly compute $\mathbf{w}_p = \mathbf{C}^T\mathbf{x}_p$ and simplify the objective function to

$$g(\mathbf{C}) = \frac{1}{P} \sum_{p=1}^{P} \|\mathbf{C}\mathbf{C}^T\mathbf{x}_p - \mathbf{x}_p\|_2^2$$

Proof:

Assuming $\mathbf{C}$ is orthonormal, meaning that $\mathbf{C}^T\mathbf{C} = \mathbf{I}$, the cost function can be formulated as

$$\begin{aligned}\|\mathbf{C}\mathbf{C}^T\mathbf{x}_p - \mathbf{x}_p\|_2^2 &= \mathbf{x}_p^T\mathbf{C}^T\mathbf{C}\mathbf{C}^T\mathbf{x}_p - 2\mathbf{x}_p^T\mathbf{C}\mathbf{C}^T\mathbf{x}_p + \mathbf{x}_p^T\mathbf{x}_p \\ &\overset{\mathbf{C}^T\underline{\mathbf{C}}=\mathbf{I}}{=} -\mathbf{x}_p^T\mathbf{C}\mathbf{C}^T\mathbf{x}_p + \mathbf{x}_p^T\mathbf{x}_p = -\|\mathbf{C}^T\mathbf{x}_p\|_2^2 + \|\mathbf{x}_p\|_2^2\end{aligned}$$

Since $\mathbf{x}_p$ is fixed with respect to the optimization over the basis matrix, minimizing the mean reconstruction error is equivalent to maximizing $\|\mathbf{C}^T\mathbf{x}_p\|$.

In particular, the assumption that the basis is orthonormal allows to learn each vector independently as the objective function decouples into

$$g(\mathbf{C}) = -\frac{1}{P}\sum_{p=1}^{P}\sum_{k=1}^{K}(\mathbf{c}_k^T\mathbf{x}_p)^2$$

4.3 Optimizing a single basis vector

Assuming the dataset has been mean-centered and that the basis vector $\mathbf{c}_1$ has unit norm, the quantity

$$h(\mathbf{c}_1) = \frac{1}{P}\sum_{p=1}^{P}(\mathbf{c}_1^T\mathbf{x}_p)^2 \text{ represents the sample variance along direction } \mathbf{c}_1.$$

By stacking the data points into a matrix $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_p] \in \mathbb{R}^{N\times P}$, one can reformulate this quantity as

$$h(\mathbf{c}_1) = \frac{1}{P}\mathbf{c}_1^T\mathbf{X}\mathbf{X}^T\mathbf{c}_1 = \mathbf{c}_1^T\left(\frac{1}{P}\mathbf{X}\mathbf{X}^T\right)\mathbf{c}_1 = \mathbf{c}_1^T\mathbf{\Sigma}\mathbf{c}_1$$

In this setting, $\mathbf{\Sigma}$ is known as the "empirical covariance matrix" and, by describing how features vary together, it provides information about the shape of the data cloud through the eigenvectors, which indicate the principal directions in which data spreads the most, and the eigenvalues, which explain the variance along these directions.

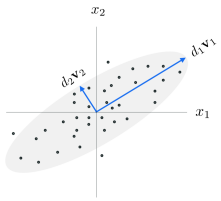


Figure 4.1: Principal component analysis as a dataset rotation.

Therefore, $h(\mathbf{c}_1)$ will be maximized when $\mathbf{c}_1 = \mathbf{v}_1$ is the eigenvector associated to the largest eigenvalue λ_1 of the covariance matrix.

More specifically, the scaled eigenvector is known as the "first principal component" of the dataset and it explains a proportion of the variance equal to

$$EVR_1 = \frac{\lambda_1}{\sum_{n=1}^{N} \lambda_n}$$

Example: Example of principal component analysis in $\mathbb{R}^2$

Consider the following dataset:

$$\mathbf{x}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \qquad \mathbf{x}_2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \qquad \mathbf{x}_3 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Start by computing the sample mean $\overline{\mathbf{x}}$ to obtain the mean-centered data points $\mathbf{c}_p = \mathbf{x}_p - \overline{\mathbf{x}}$:

$$\overline{\mathbf{x}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \Rightarrow \mathbf{c}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \qquad \mathbf{c}_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \qquad \mathbf{c}_3 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

This means that the empirical covariance matrix will be given by

$$\mathbf{\Sigma} = \frac{1}{3} \sum_{p=1}^{3} \mathbf{c}_p \mathbf{c}_p^T = \begin{pmatrix} \frac{2}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} \end{pmatrix}$$

In particular, the eigenvalues of the covariance matrix can be found by solving

$$\begin{aligned} det(\mathbf{\Sigma} - \lambda \mathbf{I}) &= det \begin{pmatrix} \frac{2}{3} - \lambda & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} - \lambda \end{pmatrix} \\ &= \left(\frac{2}{3} - \lambda \right)^2 - \frac{1}{9} = \frac{4}{9} - \frac{4\lambda}{3} + \lambda^2 - \frac{1}{9} = \lambda^2 - \frac{4\lambda}{3} + \frac{1}{3} = 0 \end{aligned}$$

$$\Delta = \frac{4}{9} > 0 \Rightarrow \lambda_{1,2} = \frac{\frac{4}{3} \pm \frac{2}{3}}{2} \Rightarrow \lambda_1 = 1, \quad \lambda_2 = \frac{1}{3}$$

Using these eigenvalues, solving $(\mathbf{\Sigma} - \lambda \mathbf{I})\mathbf{X} = 0$ allows to find the following eigenvectors:

$\mathbf{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, which corresponds to principal component $\mathbf{v}_1 = \begin{pmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{pmatrix}$

$\mathbf{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, which corresponds to principal component $\mathbf{v}_2 = \begin{pmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{pmatrix}$

In particular, the proportion of data variance explained by the two components will be

$$EVR_1 = \frac{\lambda_1}{\lambda_1 + \lambda_2} = 0.75 \text{ and } EVR_2 = \frac{\lambda_2}{\lambda_1 + \lambda_2} = 0.25$$

It should be noticed that the same reasoning can be applied to the other basis vectors when analysing the other principal components.

4.3.1 Eigenvalue decompositions

For a real-valued and symmetric covariance matrix $\mathbf{\Sigma}$, one can compute an orthonormal matrix $\mathbf{V} = [\mathbf{v}_1, \ldots, \mathbf{v}_d]$, which is obtained by stacking the eigenvectors of the covariance matrix, and a diagonal matrix $\mathbf{\Lambda}$, which contains the eigenvalues of the covariance matrix, to formulate

$$\mathbf{\Sigma} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T, \text{ implying that } \mathbf{\Sigma} \text{ is always positive semi-definite.}$$

4.4 Applications of principal component analysis

Principal component analysis can be used as an unsupervised learning algorithm that allows to compress high-dimensional data onto a lower-dimensional space, such as a linear manifold, by using the $k << d$ most relevant components to create uncorrelated features that can be exploited to simplify learning tasks or discover data patterns.

Chapter 5

Linear models

5.1 The risk minimization problem in supervised learning

Assuming data is generated by an unknown function f, a generic machine learning algorithm uses a dataset Z to output a model f^* that best "approximates" the input data.
Therefore, a machine learning algorithm aims to find an approximation $f \approx f^*$ by knowing that f^* belongs to some model family F.
From a broader perspective, a supervised learning problem provides a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where each data point has been sampled independent and identically distributed from an unknown distribution $p(\mathbf{x}, y)$, and introduces a loss function that gives the model a penalty if it uses a predicted value instead of the true label.
For this reason, the goal of a machine learning problem can be formulated as finding a function that can model the training data and generalize to make predictions on unseen data, resulting in

$$f^* = \arg\min_{f \in F} \mathbb{E}_{p(\mathbf{x},y)}[\mathcal{L}(f(\mathbf{x}), y)]$$

However, since the true probability distribution is unknown, the true average must be approximated using the sample average of the training data, meaning that

$$f^* = \arg\min_{f \in F} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f(\mathbf{x}_i), y_i)$$

5.2 Linear regression

Linear regression is a supervised machine learning algorithm that models a linear relationship between the input data and a (scalar) target variable using a linear function

$$f_\beta(\mathbf{x}) = \beta^T \mathbf{x} = \begin{pmatrix} \beta_0 & \dots & \beta_d \end{pmatrix} \begin{pmatrix} \mathbf{x}_0 \\ \vdots \\ \mathbf{x}_d \end{pmatrix} = \sum_{i=0}^{d} \beta_i \mathbf{x}_i, \text{ with } \mathbf{x}_0 = 1 \text{ to treat } \beta_0 \text{ as an intercept.}$$

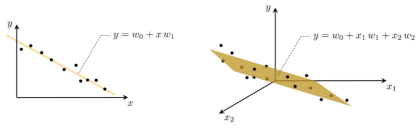


Figure 5.1: Visualization of a linear regression model.

Knowing that $y_i \approx \beta^T \mathbf{x}_i$ if the residual $(y_i - \beta^T \mathbf{x}_i)^2$ is small enough, it is possible to evaluate the quality of a linear regression model using the mean squared error metric given by

$$MSE(\beta, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \beta^T \mathbf{x}_i)^2$$

Therefore, given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the linear regression algorithm computes the parameters

$$\beta^* = \arg\min_{\beta \in \mathbb{R}^d} \mathcal{L}(\beta, \mathcal{D}) = \arg\min_{\beta \in \mathbb{R}^d} \frac{1}{N} \sum_{i=1}^{N} (y_i - \beta^T \mathbf{x}_i)^2, \text{ resulting in } f_{\beta^*}(\mathbf{x}) = \beta^{*T} \mathbf{x}.$$

These parameters are then optimized using gradient descent to minimize the mean squared error

$$MSE(\hat{\beta}_0, \hat{\beta}_1) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^{N} (y_i - (\hat{\beta}_0 + \hat{\beta}_1 \mathbf{x}_i))^2$$

$$\frac{\partial MSE}{\partial \hat{\beta}_0} = -\frac{2}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i) \quad \frac{\partial MSE}{\partial \hat{\beta}_1} = -\frac{2}{N} \sum_{i=1}^{N} \mathbf{x}_i (y_i - \hat{y}_i)$$

This means that the intercept $\hat{\beta}_0$ and the slope $\hat{\beta}_1$ are iteratively updated as follows:

$$\begin{cases} \hat{\beta}_0 = \hat{\beta}_0 - \alpha \left(-\frac{2}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i) \right) \\ \hat{\beta}_1 = \hat{\beta}_1 - \alpha \left(-\frac{2}{N} \sum_{i=1}^{N} \mathbf{x}_i (y_i - \hat{y}_i) \right) \end{cases}, \text{ going on until convergence.}$$

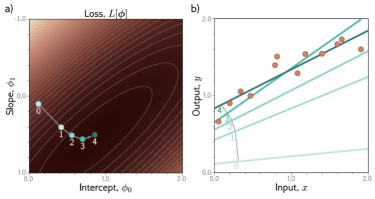


Figure 5.2: Gradient descent on a linear regression model.

Common gradient descent implementations include batch gradient descent, which performs updates on the entire dataset to ensure stable gradient convergence, or stochastic gradient descent, which performs updates on a single data point to avoid errors during convergence.
However, since these strategies often result in performance degradation or computational waste, most machine learning algorithms make use of mini-batch gradient descent, which consists of performing updates using a small subset of the dataset to ensure better efficiency and stability.

5.2.1 Alternative performance metrics

Other common performance metrics used to evaluate a linear regression model include the mean absolute error and the mean relative error, which are defined as follows:

$$MAE(\beta, \mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \quad MRE(\beta, \mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}\frac{|y_i - \hat{y}_i|}{|y_i|}$$

Alternatively, one can opt for the determination coefficient, which describes how well the model is able to explain the variance of the data points by computing

$$R^2 = 1 - \frac{MSE}{Var(y)}$$

5.2.2 Vectorized linear regression

The linear regression algorithm can be made more efficient by stacking the data points and labels into matrices.
Using this formulation, the mean squared error loss function is expressed as

$\mathcal{L}(\beta, \mathcal{D}) = \frac{1}{N}\|\mathbf{Y} - \mathbf{X}\beta\|_2^2$, which can be expanded to $N\mathcal{L}(\beta, \mathcal{D}) = \mathbf{Y}^T\mathbf{Y} - 2\mathbf{Y}^T\mathbf{X}\beta + \beta^T(\mathbf{X}^T\mathbf{X})\beta$

In particular, the gradient of the loss function will be given by

$$\nabla_\beta \mathcal{L}(\beta, \mathcal{D}) = -\frac{2}{N}\mathbf{X}^T\mathbf{Y} + \frac{2}{N}\mathbf{X}^T\mathbf{X}\beta$$

Therefore, performing gradient descent on the loss function allows to find the following closed-form solution for the optimal parameters:

$$\nabla_\beta \mathcal{L}(\hat{\beta}, \mathcal{D}) = 0 \Leftrightarrow \mathbf{X}^T\mathbf{X}\hat{\beta} = \mathbf{X}^T\mathbf{Y} \Leftrightarrow \hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

While this optimization approach is more efficient, the returned solution will be unique if and only if $\mathbf{X}^T\mathbf{X}$ is invertible.
In fact, the solution is not unique whenever there are fewer data samples than features or in the case of multicollinearity, which happens when one feature can be expressed as a linear combination of other features and the corresponding data matrix will be rank-deficient.

Example: Fitting a linear regression model using the normal equation

Consider the dataset $\mathcal{D} = \{(1,2),(2,3),(3,5)\}$ as a design matrix

$$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \text{ where the first column acts the intercept component.}$$

It is possible to fit a linear model $y = a + bx$ by solving the normal equation $(\mathbf{X}^T\mathbf{X})\hat{\beta} = \mathbf{X}^T\mathbf{Y}$:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} \hat{\beta} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$
$$\begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 3a + 6b \\ 6a + 14b \end{pmatrix} = \begin{pmatrix} 10 \\ 23 \end{pmatrix}$$

Solving this system allows to retrieve the model $\hat{y} = \frac{1}{3} + \frac{3}{2}x$.

Overall, one of the main drawbacks of vectorized linear regression lies in the fact that finding the closed-form solution is computationally expensive as the overall complexity amounts to $O(d^3)$ time.

5.3 Polynomial regression

Assuming there is some "prior knowledge" about the function to approximate, it is possible to define the model family starting from a feature map $\phi : \mathbb{R} \to \mathbb{R}^m$ that is defined as

$$\phi(x) = \begin{pmatrix} x \\ \vdots \\ x^m \end{pmatrix} \Rightarrow f_\beta(x) = \beta_0 + \beta^T\phi(x) = \sum_{i=0}^{m} \beta_i x^i$$

Alternatively, it is also possible to use a generalized feature map $\phi : \mathbb{R}^n \to \mathbb{R}^m$ that allows to model the interactions between different components of the data cloud.
While polynomial regression provides a flexible strategy for modelling data relationships, it can suffer from severe performance degradation.
In fact, using a high-degree polynomial makes the model prone to overfitting, meaning that it will perform well on the training data but fail on unseen data, whereas opting for a low-degree polynomial makes the model prone to underfitting, meaning that it will not be able to learn due to a bias.

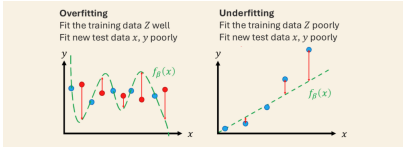


Figure 5.3: Overfitting and underfitting under a standard setting.

5.4 Linear classification models

Linear models can also be used for classification tasks that involve predicting a binary variable $y_i \in \{0, 1\}$, typically predicting $y_i = 1$ if $\beta^T \mathbf{x}_i \geq 0$.

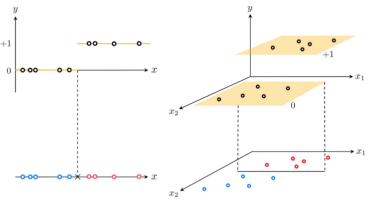


Figure 5.4: Visualization of a linear classification model.

Assuming that, for each data point, the corresponding label is drawn independent and identically distributed from a conditional distribution $p_\beta(y_i|\mathbf{x}_i)$, it is possible to formulate the likelihood of a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ as

$$L(\beta, \mathcal{D}) = p_\beta(\mathbf{Y}|\mathbf{X}) = \prod_{i=1}^{N} p_\beta(y_i|\mathbf{x}_i)$$

However, since maximizing the likelihood of a dataset tends to be a problematic task, it is typically more convenient to minimize its negative log-likelihood, which is instead defined as

$$\ell(\beta, \mathcal{D}) - \log(L(\beta, \mathcal{D})) = -\sum_{i=1}^{N} \log(p_\beta(y_i|\mathbf{x}_i))$$

Under this setting, each observation is often assumed to be drawn from a Gaussian distribution that is centered around a linear function of $\mathbf{x}$ and has unit variance, meaning that

$$p_\beta(y_i|\mathbf{x}_i) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(\beta^T \mathbf{x}_i - y_i)^2} \ \forall \mathbf{x}_i \in \mathcal{D}, \text{ resulting in}$$

$$\ell(\beta, \mathcal{D}) = -\sum_{i=1}^{N} \log(p_\beta(y_i|\mathbf{x}_i)) = \frac{N \log(2\pi)}{2} + \frac{1}{2} \sum_{i=1}^{N} (\beta^T \mathbf{x}_i - y_i)^2$$

Therefore, the goal is to express the target model as the prediction of y that is most likely to occur given an input $\mathbf{x}$ and a parametrization β, resulting in

$$f_\beta(\mathbf{x}) = \arg\max_y p_\beta(y|\mathbf{x}) = \beta^T \mathbf{x}$$

5.4.1 Logistic regression

When performing logistic regression, a linear model computes a scalar score $s(\mathbf{x}_i) = \beta^T \mathbf{x}_i$ that measures how much a data point $\mathbf{x}_i \in \mathbb{R}^d$ projects along the direction described by β.

Since the decision boundary occurs whenever $p_\beta(Y = 0|\mathbf{x}_i) = p_\beta(Y = 1|\mathbf{x}_i)$, which happens if $\beta^T \mathbf{x}_i = 0$, it is possible to model this boundary as a separating hyperplane that is perpendicular to β and passes through the origin.
Under this setting, the quantity $\beta^T \mathbf{x}_i$ measures a signed distance that can be normalized into a probability through the sigmoid function, which is defined as

$$p_\beta(Y = 1|\mathbf{x}_i) = \frac{1}{1 + e^{-\beta^T \mathbf{x}_i}} = \sigma(\beta^T \mathbf{x}_i)$$

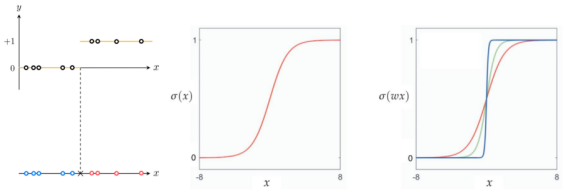


Figure 5.5: A plot of the sigmoid function.

Using the sigmoid function, it is possible to define a classifier using the following logistic model:

$$p_\beta(y|\mathbf{x}) = \begin{cases} \sigma(\beta^T \mathbf{x}) & \text{if } y = 1 \\ 1 - \sigma(\beta^T \mathbf{x}) & \text{if } y = 0 \end{cases} \qquad f_\beta(\mathbf{x}) = \begin{cases} 1 & \text{if } \sigma(\beta^T \mathbf{x}) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Knowing that $\sigma(\beta^T \mathbf{x}) \geq 0.5$ if and only if $\beta^T \mathbf{x} \geq 0$, the classifier simplifies to

$$f_\beta(\mathbf{x}) = \mathbf{1}(\beta^T \mathbf{x} \geq 0) = \begin{cases} 1 & \text{if } \beta^T \mathbf{x} \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Assuming that, for a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, each data point is drawn independent and identically distributed, its negative log-likelihood can be expressed as

$$\ell(\beta, \mathcal{D}) = -\sum_{i=1}^{N} [y_i \log(\sigma(\beta^T \mathbf{x}_i)) + (1 - y_i) \log(1 - \sigma(\beta^T \mathbf{x}_i))]$$

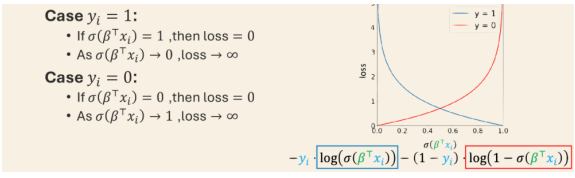


Figure 5.6: Negative log-likelihood for a logistic regression model.

Similarly for linear regression, a logistic regression model can be optimized by performing gradient descent on its negative log-likelihood:

$$\begin{aligned}\nabla_\beta \ell(\beta, \mathcal{D}) &= -\sum_{i=1}^{N} [y_i \nabla_\beta \log(\sigma(\beta^T \mathbf{x}_i)) + (1 - y_i) \nabla_\beta \log(1 - \sigma(\beta^T \mathbf{x}_i))] \\ &= -\sum_{i=1}^{N} \left[y_i \frac{\nabla_\beta \sigma(\beta^T \mathbf{x}_i)}{\sigma(\beta^T \mathbf{x}_i)} - (1 - y_i) \frac{\nabla_\beta \sigma(\beta^T \mathbf{x}_i)}{1 - \sigma(\beta^T \mathbf{x}_o)} \right] \overset{\nabla_\beta \sigma(\beta^T \mathbf{x}_i) = \sigma(\beta^T \mathbf{x}_i)(1 - \sigma(\beta^T \mathbf{x}_i)) \mathbf{x}_i}{=} \\ &= -\sum_{i=1}^{N} [y_i (1 - \sigma(\beta^T \mathbf{x}_i)) \mathbf{x}_i + (1 - y_i) \sigma(\beta^T \mathbf{x}_i) \mathbf{x}_i] = \sum_{i=1}^{N} (\sigma(\beta^T \mathbf{x}_i) - y_i) \mathbf{x}_i\end{aligned}$$

However, unlike for linear regression, no closed-form solution is available when optimizing a logistic regression model.

5.4.2 Multi-class logistic regression

When performing multi-class logistic regression on a set of labels $\mathcal{Y} = \{1, \ldots, K\}$, each class score can be computed independently using dedicated parameter vectors $\beta_1, \ldots, \beta_K$.
Under this setting, the sigmoid function can be generalized using the softmax function, which is defined as

$$p_\beta(y = j | \mathbf{x}) = \frac{e^{\beta_j^T \mathbf{x}}}{\sum_{k=1}^{K} e^{\beta_k^T \mathbf{x}}}$$

Assuming that, for a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, each data point is drawn independent and identically distributed, its likelihood can be expressed as

$$\begin{aligned}&L(\beta, \mathcal{D}) = \prod_{i=1}^{N} p_\beta(Y = y_i | \mathbf{x}_i), \text{ resulting in} \\ &\ell(\beta, \mathcal{D}) = -\sum_{i=1}^{N} \log(p_\beta(Y = y_i | \mathbf{x}_i)) = -\sum_{i=1}^{N} \left[\beta_{y_i}^T \mathbf{x}_i - \log \left(\sum_{k=1}^{K} e^{\beta_k^T \mathbf{x}_i} \right) \right]\end{aligned}$$

Using the one-hot encoding formulation $y_{ij} \in \{0, 1\}$, which denotes whether $y_i = j$, the negative log-likelihood simplifies to the cross-entropy loss between the true labels and the predicted classes, which is defined as

$$\ell(\beta, \mathcal{D}) = -\sum_{i=1}^{N} \sum_{j=1}^{K} y_{ij} \log(p(Y = j | \mathbf{x}_i))$$

At this point, it is possible to find the optimal parameters using gradient descent:

$$\nabla_{\beta_j} \ell(\beta, \mathcal{D}) = -\sum_{i=1}^{N} \mathbf{x}_i (\mathbf{1}(y_i = j) - p(Y = j | \mathbf{x}_i))$$

The optimal solution will be the same as for the binary classification problem, although each component is optimized simultaneously.

5.5 Classification metrics

Classification metrics are defined starting from the confusion matrix, which summarizes how the model's predictions compare to the ground-truth labels.

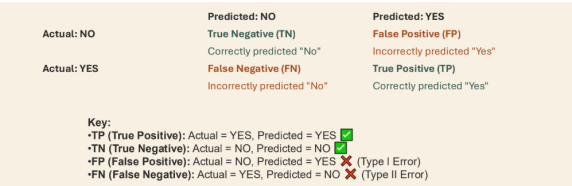


Figure 5.7: Example of a 2×2 confusion matrix.

5.5.1 The accuracy of a classification model

The accuracy of a classification model simply describes the proportion of correct predictions, meaning that it can be formulated as

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

However, this metric tends to be misleading when dealing with imbalanced datasets because a model might be able to achieve high accuracy by guessing only the samples in the majority class while failing on the data belonging to the minority class.

Example: A questionable cancer scanning model

A cancer scanning model is trained on a sample that consists of 99 healthy subjects and one sick patient.
For this reason, a model that never detects cancer will be useless, despite achieving 99% accuracy on the training dataset.

5.5.2 The sensitivity of a classification model

The sensitivity of a classification model denotes the proportion of true positives among all ground-truth positive examples, meaning that

$$Recall = \frac{TP}{TP + FN}$$

This metric is useful to monitor a model whenever a false negative is not an acceptable result.

Example: Medical screening models

For a medical screening model, not detecting a sick subject is much worse than having a healthy person undergo additional screenings.

5.5.3 The specificity of a classification model

The specificity of a classification model denotes the proportion of true negatives among all ground-truth negative examples, meaning that

$$Specificity = \frac{TN}{TN + FP}$$

This metric is useful to monitor a model whenever a false positive is not an acceptable result.

Example: Spam filtering models

For a spam filtering model, not blocking a few spam messages is far more acceptable than wrongly marking an important message as spam.

Alternatively, it is possible to define the precision of a classification model as the proportion of true positives among all positive predictions, meaning that

$$Precision = \frac{TP}{TP + FP}$$

5.5.4 The F1 score of a classification model

The F1 score of a classification model acts as a harmonic mean between its precision and recall, meaning that it is defined as

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

This metric comes in handy to assess model performance on imbalanced datasets because the model will be considered well-performing only if it can correctly classify and detect positive cases.

5.5.5 The receiving operating characteristic curve of a classification model

The receiving operating characteristic curve is a graph that describes how the performance of a classification model varies as the classification threshold is changed.
In particular, this graph is often summarized using the "area under the curve", which describes the model's overall ability, regardless of the classification threshold.

5.6 The bias-variance trade-off

Training a model typically involves splitting a dataset into a training dataset, which is used to fit the linear regression model to obtain the parameters $\hat{\beta}$, and a testing dataset, which is used to assess the model's performance on unseen data.
Assuming k different datasets are drawn from the same underlying distribution and each dataset is used to fit a model $f_{\beta_1}, \ldots, f_{\beta_k}$, model performance can also be evaluated in terms of variance, which describes how much the fitted functions vary between each other on average, and bias, which describes how much the average of the fitted functions deviates from the true function.

Example: Various degrees of bias and variance

The function $f(x) = x + 2$ generates labels with uniform measurement noise in $[-1, 1]$.
For $k \to +\infty$ training samples of six points each, the following fits are observed:

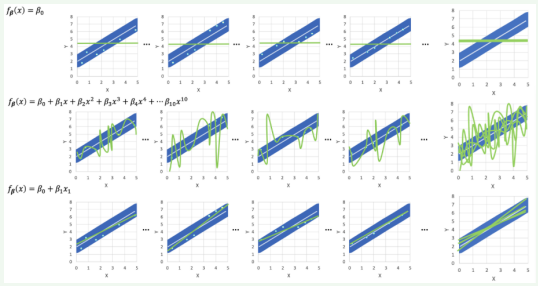


Figure 5.8: Results of using different models for fitting the same data.

The first model appears to underfit the data as each fit is identical and far from the true function, whereas the second model seems to overfit the data as the fits are very different but their average is close to the true function.
By contrast, fitting the data using a linear function yields the best results as the fits are quite similar and their average does resemble the true function.

For a linear model, the expected mean squared error can be decomposed into bias-variance terms and an "irreducible" error that comes from the data generation process, resulting in

$$\mathbb{E}[\mathcal{L}(f(\mathbf{x}), y)] = \mathbb{E}[(y - f(\mathbf{x}))^2] = (\mathbb{E}[\hat{f}(\mathbf{x}) - f(\mathbf{x})])^2 + \mathbb{E}[(\hat{f}(\mathbf{x}) - \mathbb{E}(\hat{f}(\mathbf{x})))^2] + \sigma^2$$

Assuming the chosen model belongs to the family of functions $y = f(\mathbf{x}) + \varepsilon$, where the error term ε follows a zero-mean distribution with variance σ^2, the mean squared error can be reformulated as

$$\begin{aligned} MSE &= \mathbb{E}[((f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})]) + (\mathbb{E}[\hat{f}(\mathbf{x})] - \hat{f}(\mathbf{x})) + \varepsilon)^2] \\ &= \mathbb{E}[(f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})])^2] + \mathbb{E}[(\mathbb{E}[\hat{f}(\mathbf{x})] - \hat{f}(\mathbf{x}))^2] + \mathbb{E}[\varepsilon^2] \\ &= Bias^2 + Variance + \sigma^2 \end{aligned}$$

5.7 Model regularization

As the dataset becomes larger, the error becomes bias-dominated, allowing to use lower-bias models to fit the data.
However, to avoid overfitting the data, the model can be "forced" to produce a simpler fit by introducing a data-independent regularization term that punishes complicated models, meaning that the loss function can be reformulated as

$$\mathcal{L}'(\beta, \mathcal{D}) = \mathcal{L}(\beta, \mathcal{D}) + \lambda \mathcal{R}(\beta), \text{ with } \lambda \geq 0.$$

Since the hyperparameter λ models the strength of the regularization term, it has to be chosen carefully in order to avoid overfitting.

For this reason, it is common practice to use a separate validation dataset for selecting the best hyperparameter, which is then used to re-train the model.

5.7.1 Ridge regression

Ridge regression is a modified form of linear regression that uses the following loss function:

$$\mathcal{L}(\beta, \mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \beta^T \mathbf{x}_i)^2 + \lambda\|\beta\|_2^2 = \frac{1}{N}\sum_{i=1}^{N}(y_i - \beta^T \mathbf{x}_i)^2 + \lambda\sum_{j=1}^{d}\beta_j^2$$

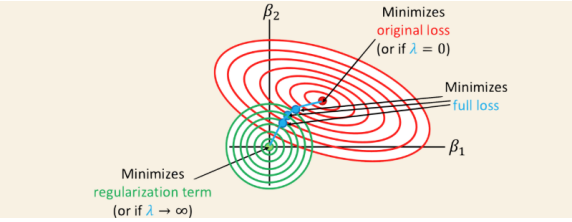


Figure 5.9: The effect of L2 regularization during optimization.

From a geometric standpoint, this regularization technique corresponds to a Gaussian prior on the model's weights because, by shrinking the coefficients towards zero, it creates a simple and more stable model.

5.7.2 Lasso regression

Lasso regression is a modified form of linear regression that uses the following loss function:

$$\mathcal{L}(\beta, \mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \beta^T \mathbf{x}_i)^2 + \lambda\|\beta\|_1 = \frac{1}{N}\sum_{i=1}^{N}(y_i - \beta^T \mathbf{x}_i)^2 + \lambda\sum_{j=1}^{d}|\beta_j|$$

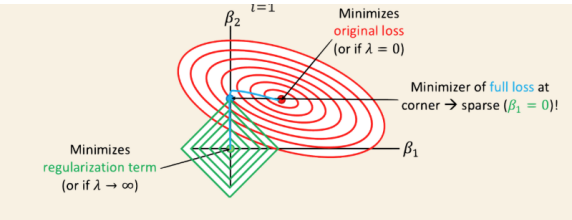


Figure 5.10: The effect of L1 regularization during optimization.

This regularization technique creates a sparse model that, by dropping irrelevant features, requires less parameters during computations.

Chapter 6

Non-parametric models

6.1 The non-parametric learning paradigm

While parametric models assume a fixed functional form f_β and focus on learning a finite number of parameters, non-parametric models do not assume a fixed form and can be used to represent arbitrarily flexible functions.

6.2 The K-nearest neighbours algorithm

Starting from a training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the K-nearest neighbours algorithm aims to classify a new data point $\mathbf{x}$ based on geometric proximity.
In fact, this algorithm first finds the K training data points that are closest to $\mathbf{x}$ and then predicts the class label through a majority among these neighbours, meaning that

$$i_1, \dots, i_K = K\text{-}\arg\min_{i \in \{1,\dots N\}} \text{dist}(\mathbf{x}, \mathbf{x}_i), \text{ from which } y = \text{Mode}(y_{i_1}, \dots, y_{i_K})$$

From a technical standpoint, the K-nearest neighbours algorithm has no explicit training phase and, rather than minimizing a loss function for optimization, model performance is evaluated using dedicated metrics.

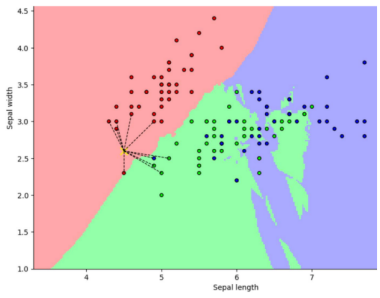


Figure 6.1: Performing K-nearest neighbours on the IRIS Dataset.

Overall, this algorithm provides a simple and interpretable framework that is efficient for dynamic datasets, although it tends to be computationally expensive and sensitive to noise coming from irrelevant features or extreme values.
Regarding the bias-variance trade-off, choosing a low value for K makes the model flexible but prone to overfitting, whereas opting for a high value for K makes the model smooth but prone to underfitting.

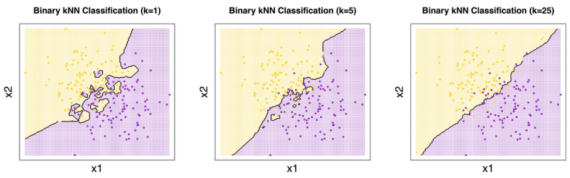


Figure 6.2: Results of K-nearest neighbours using different choices of K.

For this reason, it is common practice to introduce a validation dataset to analyse how model performance varies as K changes from 1 to $\sqrt{N}$.

6.3 Decision trees

A decision tree is a non-parametric algorithm that defines complex decision boundaries by learning simple and interpretable rules.
In fact, a decision tree is implemented as a binary tree that contains decision nodes, where a Boolean condition is used to compare a single feature x_j and a threshold t, and terminates with leaf nodes, which represent the final prediction.
Since finding an optimal decision tree is a **NP**-complete problem, decision trees are built using a top-down greedy heuristic rule that locally minimizes impurity upon each split.
From a broader perspective, the impurity of a node Z, where a set of classes $\{1, \dots, C\}$ follows a distribution where p_c denotes the proportion of data points belonging to class c, can be measured using one of two possible metrics:

1. **Gini impurity:** This metric denotes the probability of incorrectly classifying a randomly chosen data point based on the node's class distribution, resulting in

$$I_{Gini}(Z) = 1 - \sum_{c=1}^{C} p_c^2$$

2. **Entropy:** This metric quantifies the average uncertainty in the node's class distribution, meaning that

$$I_{Entropy}(Z) = -\sum_{c=1}^{C} p_c \log(p_c)$$

In particular, the quality of a decision tree can be evaluated in terms of information gain, which measures the reduction in impurity upon a split: assuming a node Z, which contains n data points, is split into the nodes Z_L and Z_R, which contain n_L and n_R nodes respectively, the resulting information gain is given by

$$IG(Z, \text{split}) = I(Z) - \left(\frac{n_L}{n} I(Z_L) + \frac{n_R}{n} I(Z_R) \right)$$

At each level, a decision tree finds the best split by sorting unique feature values $v_1, \ldots, v_m$ and tests candidate thresholds $t \in \left\{ \frac{v_i + v_{i+1}}{2} \right\}_{i=1}^{m-1}$ in order to greedily select the feature-threshold pair that maximizes the information gain.
Overall, concerning the bias-variance trade-off, implementing a shallow decision tree results in high bias and makes the model prone to underfitting, whereas opting for a deep decision tree makes the model prone to overfitting due to high variance.
However, it is possible to mitigate the risk of overfitting by introducing a regularization term or by performing post-pruning, which consists of recursively replacing decision nodes with leaf nodes and evaluating performance improvements using a validation dataset.

6.3.1 Using decision trees for regression tasks

While decision trees are often used for classification tasks, they can be adapted to perform regression by predicting the average value of the target variable, resulting in

$$\hat{y}_i = \mu_Z = \frac{1}{n} \sum_{i=1}^{n} y_i, \text{ for a node } Z \text{ with } n \text{ data points.}$$

Therefore, to achieve homogeneity during predictions, regression trees measure impurity using the mean squared error metric, meaning that

$$MSE(Z) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \mu_Z)^2$$

6.4 Ensemble learning techniques

Ensemble learning is a strategy that consists of training different tree-based models $f_1, \ldots, f_k$ and combining their results, often using a (weighted) average or majority vote, into a single, stronger model F.
Generally speaking, an ensemble model is built using the "mixture of experts" technique, which introduces a gating network $g(\mathbf{x})$ that produces a dynamic sequence of weights $\alpha_1, \ldots, \alpha_k$ for each data point, resulting in

$$F(\mathbf{x}) = \sum_{i=1}^{k} \alpha_i(\mathbf{x}) \cdot f_i(\mathbf{x})$$

Therefore, a mixture of experts model aims to learn which base model should be trusted based on the observed input $\mathbf{x}$.

6.4.1 Bagging techniques

A bagging technique is designed to reduce variance and prevent overfitting by averaging many unbiased models $f_1, \ldots, f_k$ into a more robust model F.
Starting from a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, a bagging algorithm creates k bootstrap datasets by sampling N training data points with replacement, meaning that each bootstrap dataset will omit some "out-of-bag" data points.
After creating the bootstrap datasets, each base model is trained on a single bootstrap dataset, allowing each model to learn slightly different information due to data variability, and predictions are aggregated for evaluation.
For each training data point $\mathbf{x}_j$, the base models that did not see $\mathbf{x}_j$ during training generate an out-of-bag prediction, which is compared to the true label, so that the average error across all out-of-bag samples can provide a reliable estimate of the ensemble model's generalization performance.

6.4.2 Random forests

A random forest is an ensemble of decision trees that combines bagging with random feature selection for better generalization.
In fact, upon each split, the algorithm randomly samples a subset of features so that weaker features can influence the splits and produce more diverse decision trees.

6.4.3 Boosting models

Boosting is an ensemble learning technique that builds a strong predictor by sequentially combining many weak learners that are trained to correct the errors of their predecessors.
Common boosting models include:

- **Adaptive Boosting:** This model initially assigns equal importance to each training data point and, for each round, updates sample weights to give more importance to misclassified data points.
  Assuming training lasts T steps, the resulting model will be

$$F(x) = sign\left(\sum_{t=1}^{T} \beta_t f_t(\mathbf{x})\right)$$

- **Gradient Boosting:** Rather than updating weights for data points, this model sequentially builds new learners that predict the residuals of the previous learners.
  Starting from an initial model $F_0(\mathbf{x})$, a generic residual is computed as $r_t = y - F_{t-1}(\mathbf{x})$: a new weak learner $h_t(\mathbf{x})$ is trained to predict the residual so that the ensemble can be updated using a learning rate ν, resulting in

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \nu h_t(\mathbf{x})$$

  Among this family of models, XGBoost serves as an optimized and regularized implementation of the gradient boosting algorithm.

Chapter 7

Neural networks

7.1 The representation learning paradigm

While many machine learning models rely on finding and engineering custom features, finding suitable information tends to be time consuming and impractical without expert knowledge.
For this reason, the representation learning paradigm allows a model to automatically learn features from raw data.
For instance, deep learning is a representation learning method that uses a hierarchy of layers to learn various levels of information.

Example: Using a deep learning model for face recognition

A deep neural network learns how to recognize a face starting from the extraction of simple "low-level" features, such as edges or corners, that are combined into "middle-level" features, such as shapes or textures.
The shapes and textures are then combined into "high-level" object parts, which are ultimately combined into the abstract concept of a face.

Most deep learning models are based on neural networks, which typically consist of a linear transformation of the input features followed by a (non-linear) activation function.

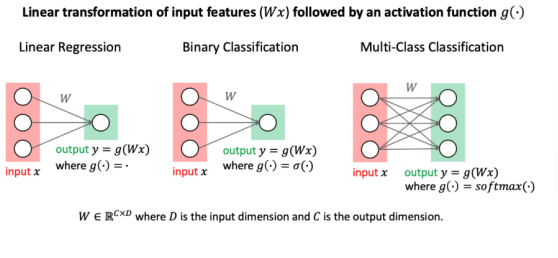


Figure 7.1: Common implementations of simple neural networks.

Overall, the capacity of a deep learning network depends on its depth, which denotes the number of hidden layers in the architecture, and its width, which indicates the number of neurons in each hidden layer.

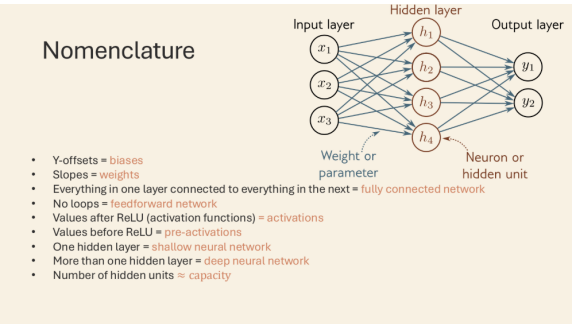


Figure 7.2: Nomenclature for a shallow neural network.

7.2 Shallow neural networks

Generally speaking, one can implement arbitrarily complex models using a "shallow" neural network because, since a stack of many linear layers is mathematically equivalent to a single linear layer, one would have

$$y = W_n(\dots(W_2(W_1x))\dots) = (W_n \dots W_2W_1)x = \mathbf{W}_{\text{stacked}}x$$

In fact, the "universal approximation theorem" states that a shallow neural network with enough hidden units can describe any continuous function on a compact subset of $\mathbb{R}^D$ to arbitrary precision. From a broader perspective, a shallow neural network with D_i inputs, D hidden units and D_o outputs can be described using the following equations:

$$h_d = Act\left(\theta_{d0} + \sum_{i=1}^{D_i}\theta_{di}x_i\right) \quad y_j = \phi_{j0} + \sum_{d=1}^{D}\phi_{jd}h_d, \text{ with } Act(z) = ReLU(z) = \max\{0, z\}.$$

Therefore, learning a shallow neural network is equivalent to learning a total of $D(D_i+1)+D_o(D+1)$ parameters, assuming the bias terms are considered.

Example: Implementing a shallow neural network based on linear regression

A shallow neural network is implemented as follows:

$$y = f(x, \phi) = \phi_0 + \phi_1 Act(\theta_{10} + \theta_{11}x) + \phi_2 Act(\theta_{20} + \theta_{21}x) + \phi_3 Act(\theta_{30} + \theta_{31}x)$$

This model corresponds to a piecewise linear function that, with the help of hidden units, can be formulated as

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3, \text{ where } h_i = Act(\theta_{i0} + \theta_i 1x).$$

In particular, learning this model is equivalent to learning a total of 20 parameters.

7.2.1 Optimizing a shallow neural network using backpropagation

A neural network is optimized by performing backpropagation, which consists of repeatedly applying the chain rule to compute the gradients of a loss function.
Generally speaking, a neural network is optimized using the (binary) cross-entropy loss function, which is defined as

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \ell_i(f(x), y), \text{ for a dataset } \mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}.$$

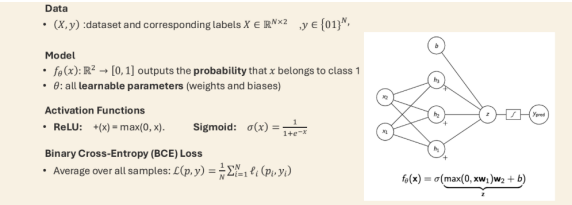


Figure 7.3: Example of a shallow neural network for binary classification.

For instance, optimizing a shallow neural network used for a binary classification task requires using the chain rule to compute the gradient of the loss function with respect to the parameters w_1, w_2 and b, resulting in

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z}\frac{\partial z}{\partial w_2} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) \cdot h = (\hat{y}-y)h^T$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z}\frac{\partial z}{\partial b} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) \cdot \mathbf{1}^T = (\hat{y}-y)\mathbf{1}^T$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z}\frac{\partial z}{\partial h}\frac{\partial h}{\partial (xw_1)}\frac{\partial (xw_1)}{\partial w_1} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) \cdot w_2^T \cdot \mathbf{1}(xw_1 > 0) \cdot x = (\hat{y}-y) \cdot w_2^T \cdot \mathbf{1}(xw_1 > 0) \cdot x$$

Part II

Network Science

Chapter 8

Understanding networks

8.1 The purpose of network science

A network is a complex system whose components interact according to three main properties:

1. **Adaptation:** The components of the system mutate and self-organize in order to maintain certain properties.
2. **Non-linearity:** A change in the input is not proportional to a change in the output.
3. **Emergence:** The system as a whole shows some properties that the individual components do not possess.

Example: Schelling's segregation model

In Schelling's segregation model, agents wish that a fraction B_a of their neighbourhood, which consists of the neighbouring agents, is from the same group.
At each round, every agent checks the fraction of matching agents B and, if $B < B_a$, it can relocate to a vacant spot, going on until every agent is satisfied.

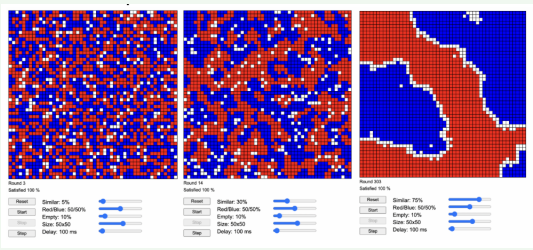


Figure 8.1: Example of Schelling's segregation model.

In particular, Schelling discovered a threshold B_{seg} such that choosing $B_a < B_{seg}$ leads to a random population configuration, whereas choosing $B_a \geq B_{seg}$ would result in segregation.

Overall, network science is useful for modelling complex systems by transforming high-dimensional data into feature representations that can be used for inference and pattern discovery.

8.2 A formal definition of networks

A network can be formally represented as a graph $G = (V, E)$ that is described using a set of nodes V and a set of edges E that connect pairs of vertices.

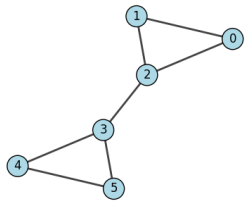


Figure 8.2: Example of a graph.

A graph is said to be "undirected" if its edges represent unordered pairs, which means that

$$E = \{\{u, v\} : u, v \in V, u \neq v\} \text{ and } \{u, v\} \in E \Leftrightarrow \{v, u\} \in E$$

By contrast, a graph is said to be "directed" if its edges indicate ordered pairs, which means that

$$E = \{(u, v) : u, v \in V, u \neq v\} \text{ and } (u, v) \in E \not\Rightarrow (v, u) \in E$$

8.2.1 Visualizing a network using an edge list

A network can be represented using an edge list in which each tuple represents a connection between two endpoints in the graph.

```
edges = [(0,1), (0,2), (1,2), (2,3), (3,4), (3,5), (4,5)]
g_py = ig.Graph(edges=edges, directed=False)
```

Listing 8.1: Creating a graph from an edge list, which requires $O(|E|)$ memory.

Alternatively, a network can be represented as a dataframe in which each row corresponds to an edge and identifies its source and target nodes.

```
edge_list_py = pd.DataFrame({
    "source": [0, 0, 1, 2, 3, 3, 4],
    "target": [1, 2, 2, 3, 4, 5, 5]
})
g_py = ig.Graph.DataFrame(edge_list_py, directed=False)
```

Listing 8.2: Creating a graph from a dataframe.

8.2.2 Visualizing a network using an adjacency matrix

It is possible to represent the connections in a network using an adjacency matrix in which $A_{ij} = 1$ if and only if $(i, j) \in E$ and $A_{ii} = 0 \; \forall i$.

```
adj_matrix = [
    [0, 1, 1, 0, 0, 0],
    [1, 0, 1, 0, 0, 0],
    [1, 1, 0, 1, 0, 0],
    [0, 0, 1, 0, 1, 1],
    [0, 0, 0, 1, 0, 1],
    [0, 0, 0, 1, 1, 0]
]
g_from_matrix = ig.Graph.Adjacency(adj_matrix, mode=ig.ADJ_UNDIRECTED)
```

Listing 8.3: Creating a graph from an adjacency matrix, which requires $O(|V|^2)$ memory.

Additionally, by considering at the definition of an adjacency matrix, it is possible to state that a graph $G = (V, E)$ is undirected if and only if its adjacency matrix is symmetric, meaning that $A^T = A$.

8.2.3 Visualizing a network using an adjacency list

It is possible to summarize the connections in a network using an adjacency list that, for each vertex in the graph, stores a list of its direct neighbours.

```
adj_list = [
    [1, 2],
    [0, 2],
    [0, 1, 3],
    [2, 4, 5],
    [3, 5],
    [3, 4]
]
```

Listing 8.4: Representing a graph using an adjacency list, which requires $O(|V| + |E|)$ memory.

It should be noticed that adjacency lists are not compatible with the `igraph` library, meaning that a conversion into an edge list or an adjacency matrix is required in order to plot a graph.

8.3 Neighbourhoods and subgraphs

On a graph $G = (V, E)$, an adjacent vertex of a node $v \in V$ is any vertex that is connected to v by an edge in the graph.
In particular, the neighbourhood of a vertex v is a subgraph composed of all the vertices adjacent to v, sometimes including v as well, and all the edges connecting these nodes.

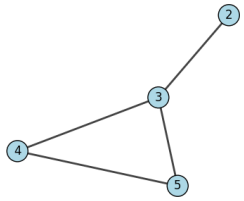


Figure 8.3: Example of a closed neighbourhood within a graph.

From a broader perspective, let $G = (V, E)$ be any graph: for a set $S \subseteq V$, the induced subgraph $G[S]$ is a graph whose vertex set is S and whose edge set consists of all the edges with both endpoints in S.

8.4 Network measures

Network measures come in handy to describe and investigate the structural properties of a network, such as connectedness or centrality.
For simplicity, it is typically assumed that networks are simple, which means that they contain neither self-loops nor multiple edges connecting the same pair of endpoints.

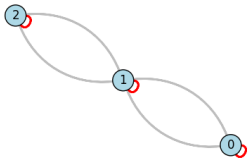


Figure 8.4: Example of a graph that is not simple.

Whenever a network is not simple, the `simplify` function can be used to turn it into a simple network by removing all redundant elements.

8.4.1 Network density

Network density provides a measure of the number of connections in a graph compared to the maximum possible number of edges.
If $G = (V, E)$ is an undirected graph, its density is given by

$$\delta = \frac{|E|}{|E_{\max}|} = \frac{|E|}{\binom{|V|}{2}} = \frac{2|E|}{|V|(|V|-1)}$$

On the other hand, if $G = (V, E)$ is a directed graph, its density is defined as

$$\delta = \frac{|E|}{|E_{\max}|} = \frac{|E|}{|V|(|V|-1)}$$

Overall, real networks tend to be sparse because, even though they consist of many nodes, they feature a very small number of edges that are typically concentrated near some hubs.

8.4.2 The degree of a node

The degree of a node denotes the number of connections that are incident to it.

- If $G = (V, E)$ is an undirected graph, the degree of a node is simply defined as

$$k_i = \sum_j A_{ij}, \text{ with } \sum_i k_i = 2|E|$$

  For this reason, the mean degree in an undirected network will be given by

$$\overline{k} = \frac{1}{|V|} \sum_i k_i = \frac{2|E|}{|V|}$$

- If $G = (V, E)$ is a directed graph, it is necessary to make a distinction between a node's in-degree, which denotes the number of incoming connections, and its out-degree, which denotes the number of outgoing connections.

$$k_i^{in} = \sum_j A_{ji} \text{ and } k_i^{out} = \sum_j A_{ij}, \text{ with } \sum_i k_i^{in} = \sum_i k_i^{out} = |E|$$
$$k_i^{tot} = k_i^{in} + k_i^{out}, \text{ with } \sum_i k_i^{tot} = 2|E|$$

  Therefore, the mean degree in a directed network will be given by

$$\overline{k^{in}} = \frac{1}{|V|} \sum_i k_i^{in} = \frac{|E|}{|V|} = \frac{1}{|V|} \sum_i k_i^{out} = \overline{k^{out}}$$

8.5 Weighted networks

A weighted network is defined as a graph $G = (V, E, w)$, where the function $w : E \to \mathbb{R}^+$ assigns a (positive) weight to each edge in the network.

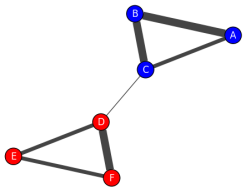


Figure 8.5: Example of a weighted graph.

For this reason, the adjacency matrix of a weighted network can be defined as

$$W_{ij} = \begin{cases} w_{ij} & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$$

In particular, if a weighted network is undirected, its adjacency matrix is guaranteed to be symmetric, although, if a weighted network is directed, its adjacency matrix may or may not be symmetric.

8.5.1 The strength of a node

The strength of a node can be seen as a generalization of the concept of degree for weighted networks. In particular, if $G = (V, E, w)$ is an undirected graph, the strength of a node is simply defined as

$$s_i = \sum_j w_{ij}$$

On the other hand, if $G = (V, E, w)$ is a directed graph, it is necessary to make a distinction between a node's in-strength and out-strength.

$$s_i^{in} = \sum_j w_{ji} \text{ and } s_i^{out} = \sum_j w_{ij}$$

8.6 The degree distribution of a network

For an undirected and unweighted graph, the degree distribution describes the frequency of node degrees within the network.
Since this distribution is typically discrete, it can be defined using raw frequencies or by normalizing frequencies into a probability mass function, which describes the probability that a node in the network has a given degree.

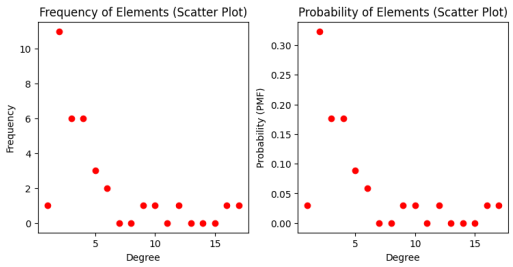


Figure 8.6: Frequencies and PMF for the degree distribution of the "Zachary Karate Club" network.

Alternatively, it is possible to study the degree distribution of a graph through its cumulative distribution function, which denotes the tail probability of the degree distribution.

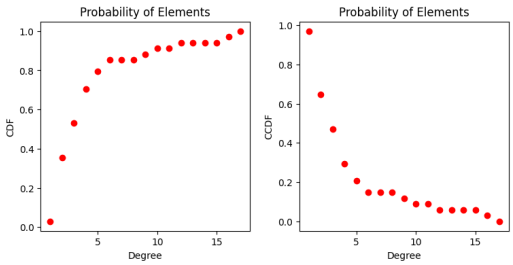


Figure 8.7: CDF and CCDF for the degree distribution of the "Zachary Karate Club" network.

In fact, using the (complementary) cumulative distribution function reveals that the (discrete) degree distribution of many real-world networks can be approximated using a continuous power law distribution that, for a constant C and an exponent $\gamma > 0$, is defined as $p(x) = Cx^{-\gamma}\mathbb{I}_{\mathbb{R}^+}(x)$.
Since the power law distribution belongs to the family of heavy-tailed distributions, its (sample) mean is often a non-representative value for the typical behaviour of the distribution.
In a more general sense, if a power law is defined over the interval $[x_{\min}, +\infty)$, its mth momentum is defined as

$$\begin{aligned}\langle x^m \rangle &= \int_{-\infty}^{+\infty} x^m p(x)\ dx \\ &= \int_{x_{\min}}^{+\infty} x^m C x^{-\gamma}\ dx = \int_{x_{\min}}^{+\infty} C x^{m-\gamma}\ dx = \frac{C}{m-\gamma+1} x^{m-\gamma+1}\Big|_{x_{\min}}^{+\infty} < +\infty \Leftrightarrow \gamma > m+1\end{aligned}$$

In particular, one can estimate the exponent of a power law distribution by fitting a linear regression model to the logarithmically transformed distribution, having

$$\log(p(x)) = \log(Cx^{-\gamma}) = \log(C) + \log(x^{-\gamma}) = \log(C) - \gamma \log(x)$$

8.7 Bipartite networks

A bipartite network consists of two distinct types of nodes, with edges connecting only nodes belonging to different groups.

Example: Using bipartite networks for recommendation systems

A bipartite network showing the users that are subscribed to certain subreddits can be used to implement a collaborative-filtering-based recommendation system that suggests new content to users based on the preferences shown by similar users.

Assuming the two node groups are represented using two disjoint sets V_i and V_j, a bipartite network can be formally represented as an undirected graph $G = (V, E)$, with $V = V_i \cup V_j$ and $E \subseteq V_i \times V_j$.

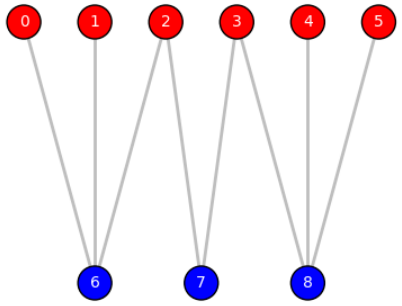


Figure 8.8: Example of a bipartite graph with $|V_i| = 6$ and $|V_j| = 3$.

In particular, a bipartite network is typically represented using an biadjacency matrix I, which has size $|V_i| \times |V_j|$, such that $I_{ij} = 1$ if and only if $\{i, j\} \in E$.

8.7.1 The density of a bipartite network

Assuming bipartite networks are undirected, the constraint that edges can only connect nodes in different partitions requires formulating the density of a bipartite network $G = (V, E)$ as

$$\delta = \frac{|E|}{|E_{\max}|} = \frac{|E|}{|V_i| \cdot |V_j|}$$

8.7.2 Performing one-mode projection on a bipartite network

While bipartite networks are useful to represent certain systems, they can also be used to build "co-occurrence" networks that contain only the nodes belonging to one of the two partitions.
This procedure, known as "one-mode projection", can be performed by computing either the matrix $A_i = II^T$, which represents the projection onto partition V_i, or the matrix $A_j = I^T I$, which represents the projection onto partition V_j.

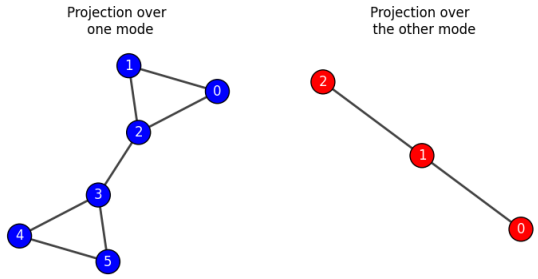


Figure 8.9: One-mode projections obtained from a bipartite network.

Example: Projecting onto users and subreddits

Consider a bipartite network where the set V_i represents Reddit users and the set V_j represents popular subreddits: two users will be connected if they are both part of some subreddit, whereas two subreddits will be connected if they share at least one subscriber.

Generally speaking, performing one-mode projection on a bipartite network always returns an undirected graph, which, however, is not guaranteed to be simple or unweighted.
However, one can retrieve the corresponding adjacency matrix by simplifying the network and, if necessary, removing the weights obtained during the projection.
Overall, the main issue of one-mode projection lies in the fact that it becomes too computationally expensive on large networks.
For this reason, common optimization strategies include block-matrix multiplication, which consists of using matrix algebra con smaller sub-matrices for which the overall computational cost is acceptable, or the clique algorithm, which treats projection as a union of cliques formed by groups of nodes that share a common neighbour from the other partition.

8.8 Paths in networks

For a network $G = (V, E)$, a path is a sequence of distinct nodes in which each node is adjacent to the next one.

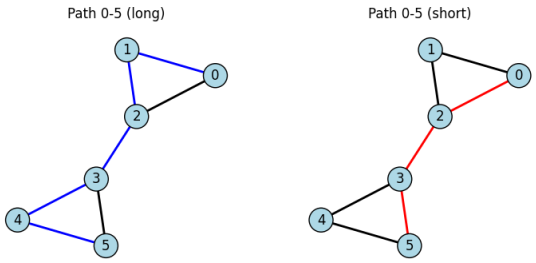


Figure 8.10: Example of paths in a graph.

Generally speaking, the length of a path is measured as the number of edges that compose it, although, for a weighted network, the sum of the edge weights is used as a measurement instead.
In particular, the distance between two nodes corresponds to the length of the shortest path between them, with $d_{i,i} = 0 \; \forall i \in V$ and $d_{i,j} = +\infty$ whenever no path connects node i to node j.
However, when considering a directed graph, the distance from node i to node j might not be equal to the distance from node j to node i as link directions must be respected.

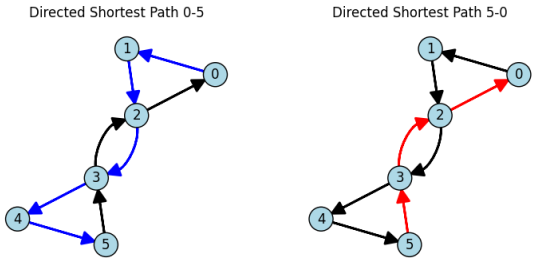


Figure 8.11: Shortest paths between two nodes in a directed graph.

From a broader perspective, distances in a network are measured using the breadth-first search algorithm, which acts as a variant of Dijkstra's algorithm for unweighted graphs.
In fact, this algorithm starts from a source node, which is at distance zero from itself, and finds all its neighbours, which are assigned distance equal to one from the source node.
The neighbours are then added to a queue, which is used to iteratively compute all the distances in the graph until all the nodes have been visited by the algorithm.

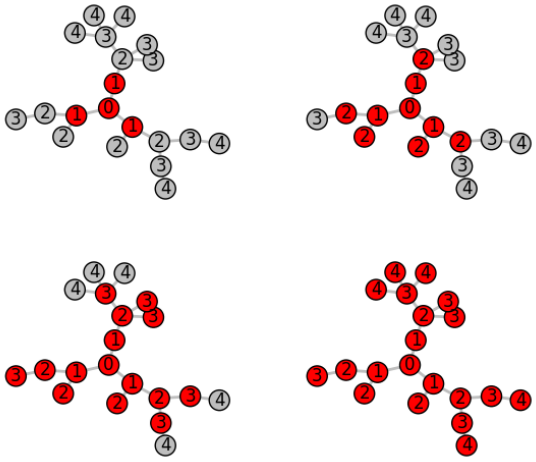


Figure 8.12: Performing breadth-first search on an undirected graph.

8.8.1 The average path length

If $G = (V, E)$ is an undirected graph, the average distance for a path is given by

$$\langle d \rangle = \frac{1}{\binom{|V|}{2}} \sum_{i \neq j} d_{i,j}$$

By contrast, if $G = (V, E)$ is a directed graph, the average distance for a path is instead defined as

$$\langle d \rangle = \frac{1}{|V|(|V|-1)} \sum_{i \neq j} d_{i,j}$$

Overall, real networks tend to exhibit the "small-world property" because, due to the presence of hubs, any node can be reached from any other node in a small number of steps, resulting in $\langle d \rangle \propto \log(|V|)$.

Example: The US Air Transportation Network

Let $G = (V, E)$ be a graph whose nodes represent airports and whose edges represent direct flights from one airport to another: the average distance between two airports in the network tends to be quite small because many airports can be connected by a path passing through some international airport, which can act as a hub for the network.

8.8.2 Connectedness in networks

Sometimes, a network simply consists of multiple components, which represent subgraphs that are disconnected from each other.
In particular, it is possible to make a distinction between "weakly connected components", where each pair of vertices is connected only if edge direction is ignored, and "strongly connected components", where each pair of vertices is connected by a directed path.

Example: The six degrees of separation

Let $G = (V, E)$ be a graph whose nodes represent scientists and whose edges connect scientists that co-authored some article: while, historically, many research fields rarely collaborated, recent developments have brought many researchers into a giant connected component that is often summarised using the Erdős number.

8.8.3 Random walks

On a graph $G = (V, E)$, a random walk is a stochastic process in which an agent starts from a source node and iteratively moves to a randomly chosen neighbour until a target node is reached.

8.9 Network descriptives

Network descriptives can be grouped into global descriptives, which capture features relating to the whole network, and local metrics, which describe features for a single node.

8.9.1 The clustering coefficient

The clustering coefficient measures node cohesiveness within a network.
Since the simplest structural graph is a triangle, the global clustering coefficient is defined as the

proportion of triangles to connected triples, meaning that

$$C = \frac{\text{Number of Triangles}}{\text{Number of Connected Triples}}$$

Alternatively, it is possible to exploit the lth power of the adjacency matrix, which indicates the number of distinct paths of length l from node i to node j, to formulate the the global clustering coefficient as

$$C = \frac{tr(A^3)}{\sum_{i \neq j} A^2_{ij}}$$

By contrast, the local clustering of a node represents the density of the subgraph formed by its neighbours, meaning that

$$c_i = \frac{2m_i}{k_i(k_i - 1)}, \text{with } c_i = 0 \text{ if } k_i \leq 1.$$

In particular, it is possible to express the global clustering coefficient of a network as the average local clustering coefficient of its nodes, resulting in

$$C = \frac{1}{|V|} \sum_{i=1}^{|V|} c_i$$

8.9.2 The centrality of a node

Centrality measures allow to identify structurally important nodes within a network.
In particular, node centrality can be evaluated according to one of the following metrics:

- **Closeness:** Closeness centrality measures how close a node is to the other nodes based on shortest paths between them.
  For this reason, closeness centrality is expressed as

  $$Closeness(i) = \frac{|V| - 1}{\sum_{j \in V \setminus \{i\}} d_{i,j}}$$

  If a network is not (strongly) connected, the harmonic closeness centrality metric considers $d_{i,j} = 0$ if no path between nodes i and j exists.

- **Betweenness:** Betweenness centrality measures the extent to which a node can act as a bridge along the shortest paths between pairs of nodes.
  Assuming σ_{st} denotes the number of shortest paths between nodes s and t, with $\sigma_{st}(i)$ indicating the number of paths passing through node i, the betweenness centrality of a node is defined as

  $$Betweenness(i) = \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}}$$

Chapter 9

Random networks

9.1 The purpose of random graph models

Random graph models come in handy to construct ensembles, which represent probability distributions over possible network configurations that feature specific structural properties encoded by fixed parameters.
In fact, one can assess the statistical significance of a property observed in a real network by comparing it against synthetic random graphs.

9.2 The Erdős-Rényi model

The Erdős-Rényi model is an ensemble $G(n, p)$ that fixes the number of nodes n and the independent probability p of having an edge between any two nodes.

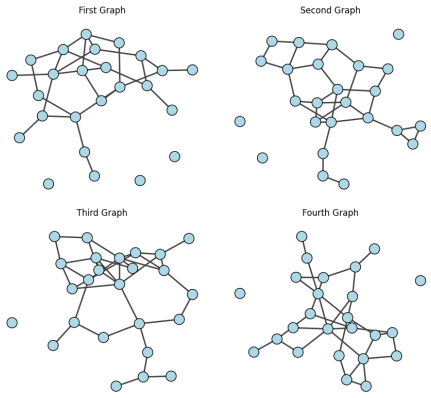


Figure 9.1: Examples of Erdős-Rényi networks from the ensemble $G(n = 25, p = 0.1)$.

Alternatively, the Erdős-Rényi model can be defined using an ensemble $G(n, m)$ that fixes the number of nodes n and the number of edges m.

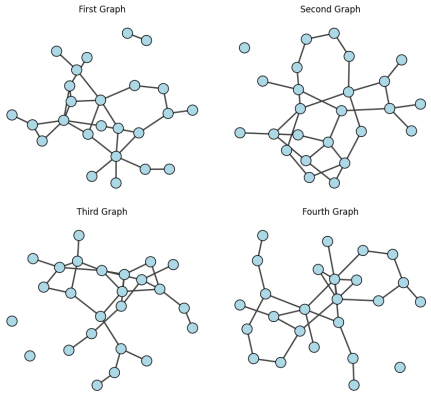


Figure 9.2: Examples of Erdős-Rényi networks from the ensemble $G(n = 25, m = 29)$.

9.2.1 The edge and degree distribution of an Erdős-Rényi network

Since edges may or may not be added through independent and identically distributed Bernoulli trials, it is possible to discover that the edges in an Erdős-Rényi network follow a binomial distribution defined by

$$\mathbb{P}(m) = p^m(1-p)^{\binom{n}{2}-m} \Rightarrow \langle m \rangle = \binom{m}{2} p$$

This phenomenon implies that the degree distribution of an Erdős-Rényi network is also binomial, meaning that

$$p_k = \binom{n-1}{k} p^k (1-p)^{n-1-k} \Rightarrow \langle k \rangle = (n-1)p$$

However, as $n \to +\infty$ for real networks, the degree distribution tends to be approximated by a Poisson distribution with rate $\lambda = \langle k \rangle = (n-1)p$, resulting in

$$p_k = \frac{e^{-\langle k \rangle} \langle k \rangle^k}{k!}$$

However, since the binomial and Poisson distributions lack heavy tails, the Erdős-Rényi model fails to reproduce the heavy-tailed degree distributions observed in real networks.

9.2.2 Analysing components in an Erdős-Rényi network

While the Erdős-Rényi model fails to capture real degree distributions, it automatically encodes the small-world property.
In fact, it is possible to show inductively that the average number of nodes located at distance s from a source node is equal to $\langle k \rangle^s$.
Therefore, assuming an Erdős-Rényi network has diameter d, it will be the case that

$$\langle k \rangle^d \approx n \Rightarrow d = \frac{\ln(n)}{\ln(\langle k \rangle)}$$

This phenomenon implies that the size of the largest connected component in an Erdős-Rényi network depends on the probability of having an edge between any two nodes.
More specifically, when $p = 0$, the network consists of a set of small isolated components that do not depend on the size of the network.
By contrast, if $p = 1$, the network acts as a single n-clique whose structure does depend on the size of the network.
Overall, for an Erdős-Rényi network, it is possible to prove that, if S is the proportion of nodes that belong to the largest connected component, the following relation holds:

$$S = 1 - e^{S\langle k \rangle} \Rightarrow \langle k \rangle = \frac{\ln(1 - S)}{S}$$